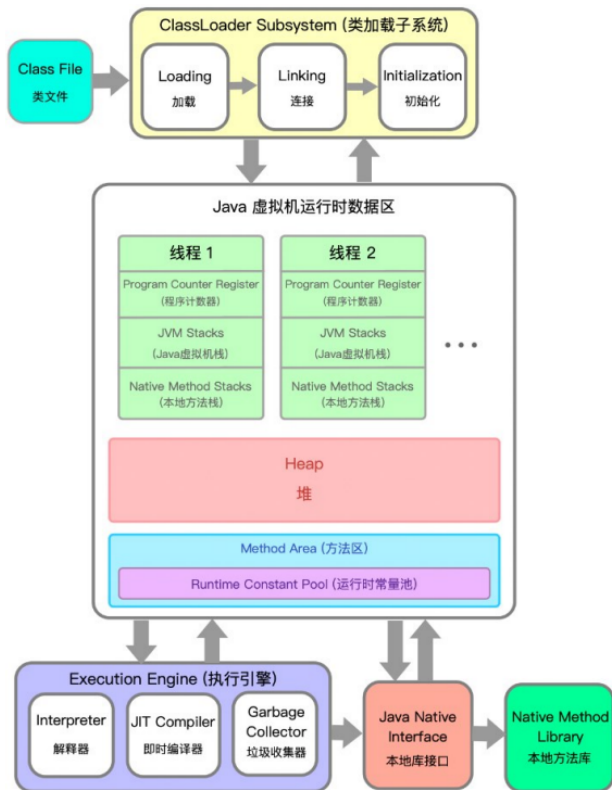
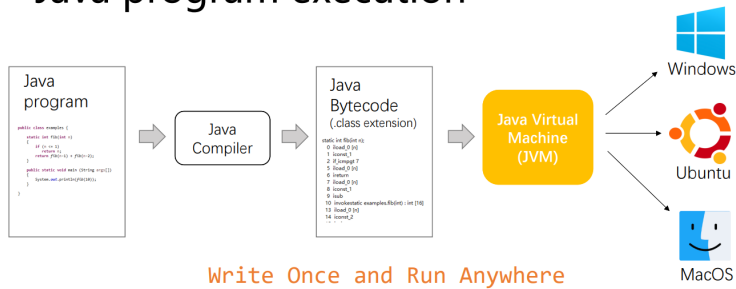


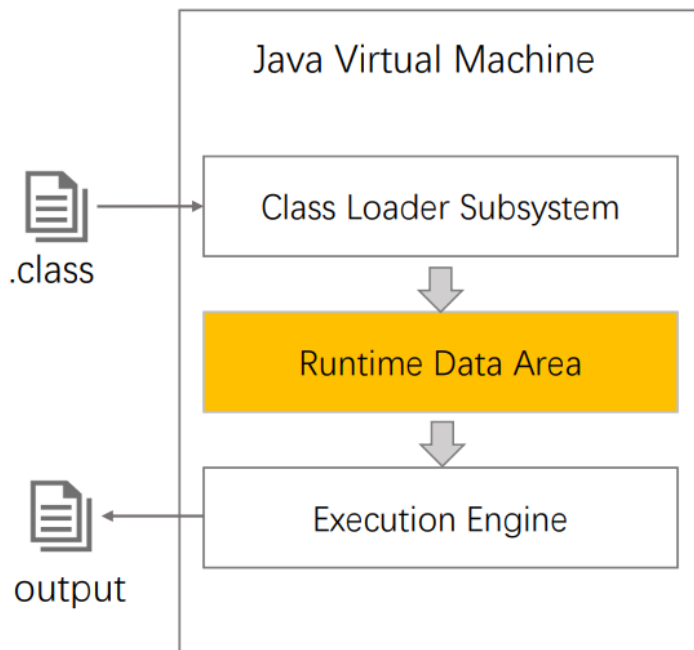
Lecture 1, 13, 14

- Java & JVM

Java program execution

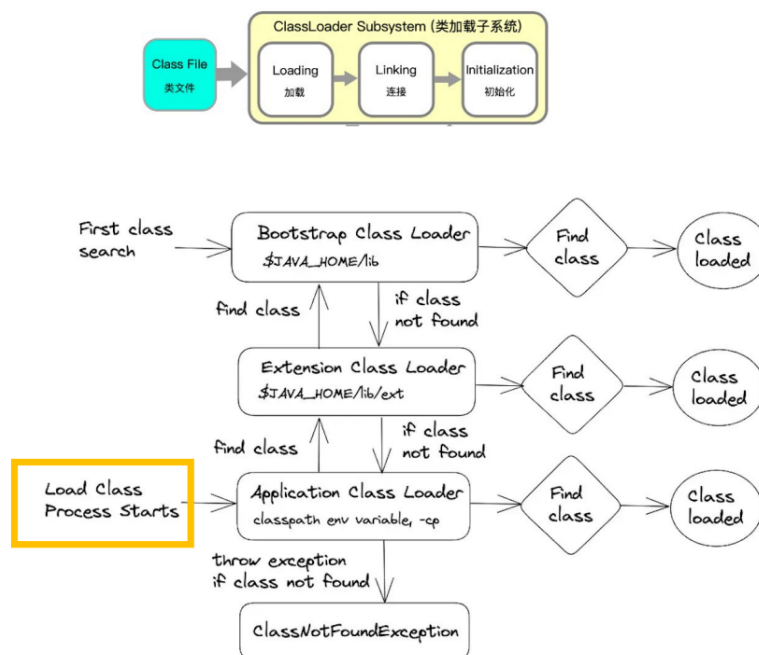


- Java Virtual Machine (JVM)



- “类加载器”：

- 定位并加载必要的 .class 或 .jar（Java 存档，.class 文件的集合）文件到内存中
 - 提供标准 Java 包的 .jar（例如 java.lang、java.io）
 - 应用程序所需的 .class 和 .jar（依赖），由 classpath 指定
- 当类加载器无法定位某个所需的 .class 时会发生错误
-

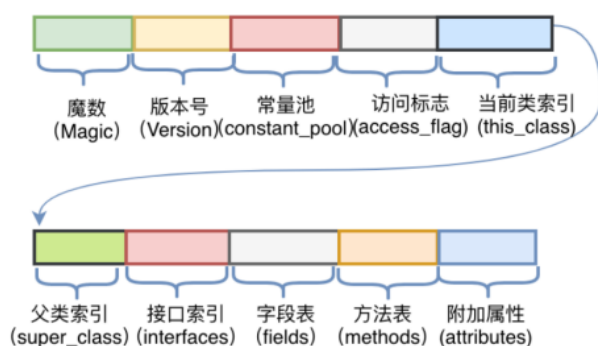


- loading

- 主要步骤：

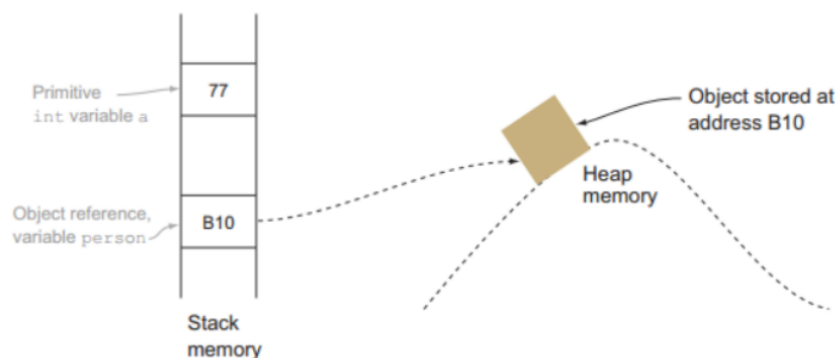
- JVM 根据类的全限定名检索该类的字节码（bytecode）。
- 在方法区（method area）中保存类的结构信息。

- JVM 创建一个 `java.lang.Class` 对象，作为运行时访问该类所有信息的入口点。
- 父委派模型说明（示意图）：
 - 当一个类加载器要加载类时，先委托父加载器尝试加载；
 - 只有父加载器无法找到类时，当前加载器才会尝试加载；
 - 常见顺序：Bootstrap ClassLoader → Extension ClassLoader → Application ClassLoader；若都找不到则抛出 `ClassNotFoundException`。
- 当类被加载后，JVM 在方法区存储下列类级别信息：

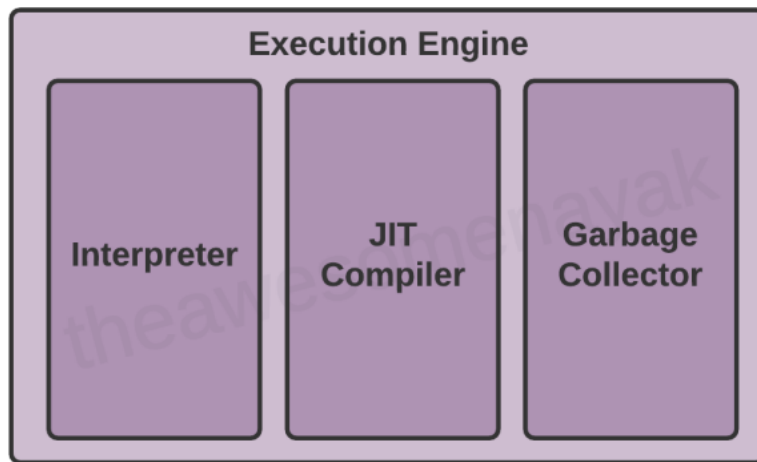


- 类信息 (Class info)：类名、访问修饰符、父类、实现的接口、静态变量等。
- 字段信息 (Field info)：字段名、类型、修饰符等。
- 方法信息 (Methods)：方法名、参数、返回类型、修饰符与方法的字节码等。
- 常量池 (Constant pool)：字符串字面量、final 常量以及对方法/字段/类的符号引用（如 `println` 的引用）。
 - 在类文件中，常量池条目类型包括 `Methodref`、`Fieldref`、`Class`、`NameAndType`、`Utf8`（字符串）等。
 - 常量池在链接/解析阶段用于将符号引用解析为直接引用；示例中展示了一个简单类（含 `System.out.println("hello word")`）对应的常量池条目（如 `println` 的 `Methodref`、`"hello word"` 的 `Utf8` 等）。
 - 常量池有助于字节码的紧凑表示与运行时的符号解析。
- linking
 - 验证 (Verify)：
 - 验证字节码是否符合 Java 规范（确保格式与结构合法、安全、不会破坏 JVM 的完整性）。
 - 准备 (Preparation)：
 - 为类的静态变量分配内存并设置默认初始值（例如数值型为 0，引用型为 `null`）。
 - 此阶段并不执行静态初始化块或赋予最终常量的运行值，只是做内存分配与默认初始化。
 - 解析 (Resolve)：

- 将常量池中的符号引用（symbolic references）解析为直接引用（direct references / 内存地址或方法表偏移）。
- 解析用于确定类或方法名对应的实际实现位置（例如找到某个方法的具体代码地址）。
- 解析也适用于 private 方法和 static 方法等需要静态绑定的引用。
- initialization
 - 为静态变量赋予程序指定的初始化值（而非准备阶段的默认值）。
 - 执行类中的静态初始化块（static blocks）。
 - 初始化通常在类第一次主动使用（如首次创建实例、首次访问静态字段或调用静态方法等）时触发。
- “运行时数据区”：
 - 存储各种数据和信息：
 - 方法区：类级别的数据
 - 堆区：对象/实例
 - 栈区：局部变量

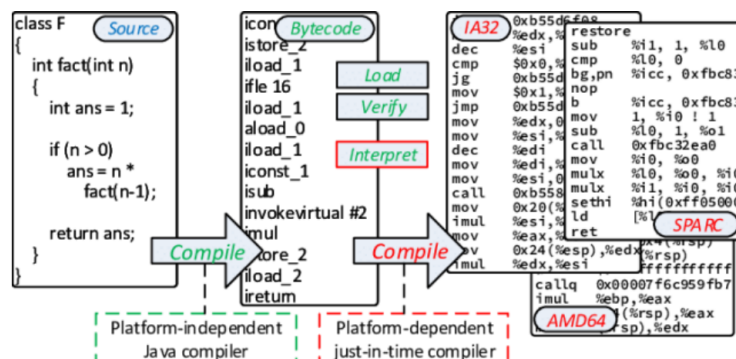


- 栈内存（Stack memory）
 - 基本类型 int 变量 a
 - 对象引用，变量 person
- 堆内存（Heap memory）
 - 对象存放在地址 B10
- “执行引擎”：
 - CPU 指令是与平台相关的：
 - 计算机的处理器（例如 Intel、AMD 或 ARM）理解机器码，而不是 Java 字节码。
 - 作用：将“可在任意平台运行”的 .class 字节码翻译为“可在该特定机器上运行”的指令。
 - JVM 以方法为基本的执行单元。从方法被调用到方法执行完成的整个过程，对应于一个栈帧被压入 JVM 栈并随后弹出的过程。



- 主流 JVM 最初使用**解释器 (Interpreter)** 来解释并执行字节码。
 - 然而，对于热点代码 (Hot Spot code) 这效率低下：
 - 被多次调用的方法
 - 经常执行的循环
- 为了提高效率，**JIT 编译器 (即时编译器)** 会将热点代码编译并优化为本地机器码。

Execution Engine: The Process



- 垃圾回收**
 - 栈区 (Stack area)：静态内存分配；一旦类信息被加载，所需内存即可计算。
 - JVM 栈
 - 局部变量表：存放方法作用域内的数据，如参数和局部变量。
 - 操作数栈：后进先出 (LIFO) 的工作区，字节码指令通过该栈推入/弹出值以完成计算。
 - 返回地址：记录被调用方法返回后，JVM 应在调用者字节码中继续执行的位置 (返回点)。
 - 动态连接：
 - 字节码中包含符号引用 (例如方法名)，栈帧持有对运行时常量池的引用。

- JVM 在运行时将这些符号引用解析为实际的方法对象（动态解析），从而支持多态。
- 示例：执行 `Animal a = new Dog(); a.speak();` 时，常量池里可能有对 `Animal.speak` 的 `methodref`；运行时通过对象 `a`（指向堆中 `Dog` 实例）确定实际类型为 `Dog`，然后使用 `Dog` 的方法表选择并调用 `Dog.speak()`。
- 堆区（Heap area）：动态内存分配；只有在运行时才知道需要创建或销毁多少对象（在程序执行之前未知）。

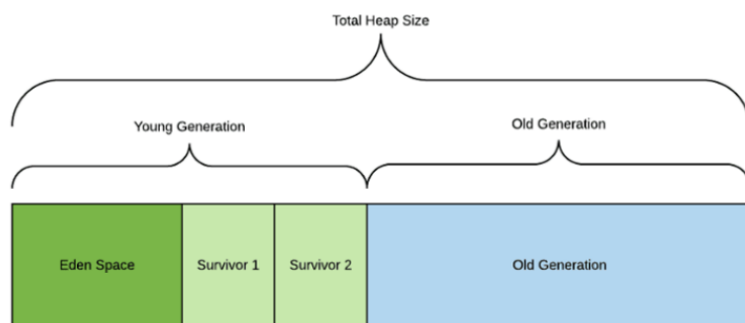


Image source: <https://backstage.forgerock.com/knowledge/kb/article/a75965340>

- 年轻代（Young Generation）是所有新对象分配并进行“成长/年龄累计”的区域（通常比老年代小）。
 - 组成（通常布局示意）：Eden 区 + 两个 Survivor 区（Survivor 1 / Survivor 2）。
 - 当年轻代填满时，会触发一次“Minor GC”（小规模回收）：
 - 将存活对象从 Eden 移到 Survivor 区，经过多次 Minor GC 后仍然存活的对象会晋升（tenure）到老年代。
- 老年代（Old Generation）用于保存长寿命（长时间存活）的对象。
 - 晋升规则：通常为年轻代对象设定一个 age 阈值，当对象在年轻代经过足够次数的 Minor GC 并达到该年龄时，就会被移动（晋升）到老年代。
 - 当老年代空间被占满或达到某些阈值时，会触发“Major GC”或“Full GC”（全堆回收），这是更昂贵的回收操作，可能会导致更长时间的停顿（停顿时间通常比 Minor GC 长）。
 - 小结：年轻代回收（Minor GC）频繁且代价较小，老年代回收（Major/Full GC）不那么频繁但代价更高；GC 策略通常以此分代设计来优化性能与暂停时间。
- 垃圾回收的关注点是堆区。
- 哪些对象是“垃圾”？
 - 引用计数算法 reference counting
 - 引用计数垃圾回收算法为每个对象关联一个引用计数。
 - 只要指向该对象的引用数量大于零，这些算法就认为该对象是存活的。

- 可达性分析算法 reachability analysis
 - 对于循环引用，用此方法
 - 从 GC 根（GC Roots）出发，迭代地查找可达对象。
 - GC 根是那些本身被 JVM 引用的对象：
 - 活动方法中的局部变量
 - 已加载类的静态变量
 - 其他对象，即使引用计数大于 0，也会被回收为垃圾。

• 测试 Testing

• 软件测试概述

- 软件测试
 - 软件测试是评估和验证软件产品或应用是否按预期工作的过程。
 - 它通过手动或自动化工具执行软件/系统组件，以评估一个或多个关注的属性。
- 软件测试的类型：
 - 单元测试（Unit Test）：在隔离环境中测试单个方法或类；单元是应用中最小的可测试组件。
 - 集成测试（Integration Test）：测试一组相关组件/类，确保它们能协同工作。
 - 系统测试（System Test）：评估完整的软件系统，确认其功能符合预期并满足规范要求。
 - 验收测试（Acceptance Test）：在完整集成的系统上进行测试，通常针对用户界面或用户场景进行验证。
 - 回归测试（Regression Test）：确保修改或新增功能不会破坏系统的既有功能或引入新缺陷。
- JUnit 5 概述：JUnit 是用于 Java 的开源单元测试框架。JUnit 5 是最新版本，使用新的 org.junit.jupiter 包来提供注解和类。JUnit 5 利用 Java 8+ 的特性（如 lambda），使测试更强大且更易维护。JUnit 5 添加了许多用于描述、组织和执行测试的实用新特性。

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class CalculatorTest {

    @Test
    void add() {
        assertEquals(4, Calculator.add(2, 2));
    }

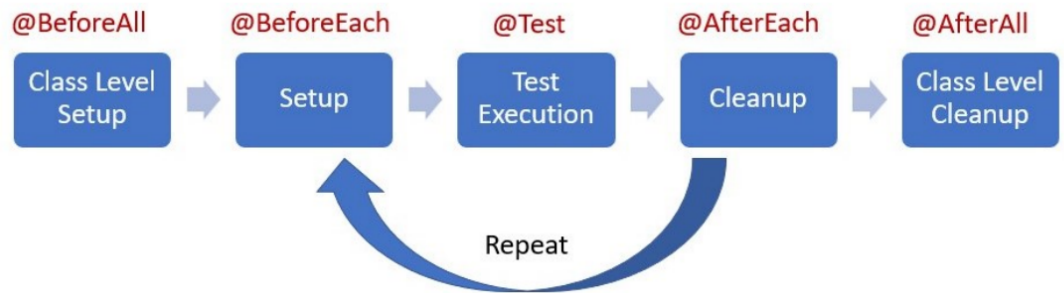
    @Test
    void multiply() {
        assertEquals(4,
            Calculator.multiply(2, 2)),
            () -> assertEquals(-4,
                Calculator.multiply(2, -2)),
            () -> assertEquals(4,
                Calculator.multiply(-2, -2)),
            () -> assertEquals(0,
                Calculator.multiply(1, 0));
    }
}
```

- 测试类 Test Class

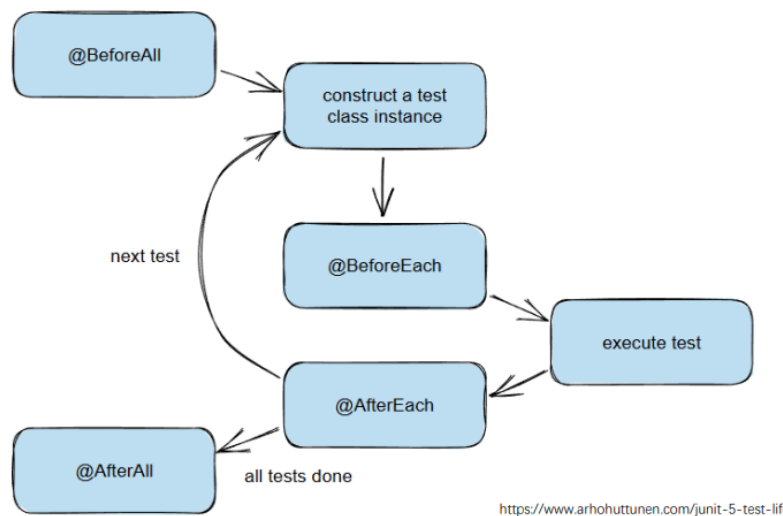
- 任何包含至少一个测试方法的类都可以是测试类。
- 测试类不得为抽象类，并且必须只有一个构造函数。
- 测试类不必须是 public，但不得为 private。
- 测试方法 Test Methods
 - 任何用注解标记的方法都可作为测试方法，例如 @Test、@RepeatedTest、@ParameterizedTest、@TestFactory、@TestTemplate 等。
 - 测试方法可以在当前测试类中声明，也可以从父类或接口继承。
 - 测试方法不得为抽象，且不得返回值（除非为 @TestFactory 等要求返回值的特殊方法）。
- 测试执行
 - 断言（Assertions）是一组用于在测试方法中验证条件的工具方法（如 assertEquals、assertAll 等）。
 - 运行某个测试类（例如 CalculatorTest）时，会执行该类中所有的测试方法并报告结果。
- 生命周期方法 Lifecycle Methods：

```
class CalculatorTest {  
    Calculator c;  
  
    @BeforeEach  
    void setUp() {  
        c = new Calculator();  
    }  
  
    @Test  
    void add() {  
        assertEquals(4, Calculator.add(2, 2));  
    }  
  
    @Test  
    void multiply() {  
        assertEquals(4, Calculator.multiply(2, 2));  
    }  
  
    @AfterEach  
    void tearDown() {  
        c = null;  
    }  
}
```

- 使用 @BeforeAll、@AfterAll、@BeforeEach、@AfterEach 标记的方法称为生命周期方法。
- @BeforeEach 表示该方法应在当前测试类中的每个 @Test 方法之前执行。
- @BeforeEach 方法必须为 void 返回类型，不能为 private，并且不能为 static。
- @BeforeAll：表示被标注的方法应在当前测试类的所有测试之前执行。
 - 与 @BeforeEach 不同，@BeforeAll 方法在给定的测试类中只会执行一次。
 - @BeforeAll 方法必须返回 void，不能是 private，并且默认情况下必须是 static。
 - 通常将数据库连接等重量级对象的创建与清理放在类级别的设置/拆除中执行（即在 @BeforeAll/@AfterAll 中处理）。
- 测试用例的完整生命周期可分为三个阶段：

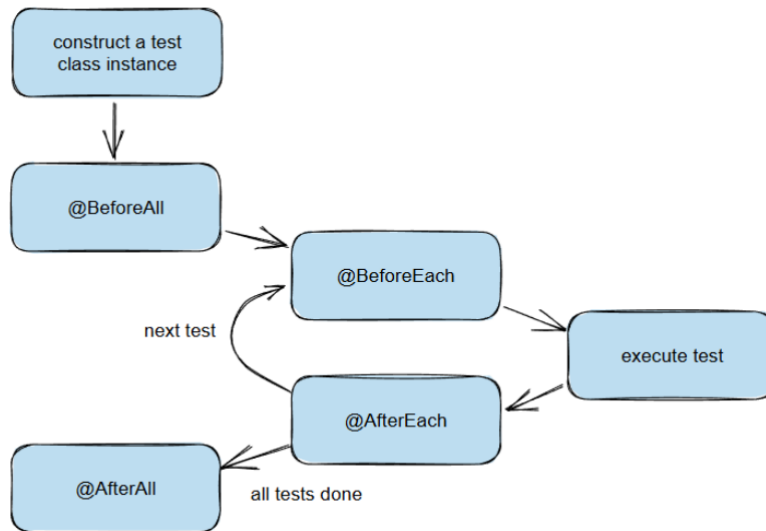


- 准备（Setup）：建立测试所需的基础设施。JUnit 提供类级别的设置（`@BeforeAll`）和方法级别的设置（`@BeforeEach`）。通常将重量级对象（如数据库连接）放在类级别创建，而将轻量级对象在方法级别重置。
- 测试执行（Test Execution）：执行测试并进行断言，结果表明测试成功或失败。
- 清理（Cleanup）：清理在第一阶段建立的测试基础设施。拆除也可在类级别（`@AfterAll`）和方法级别（`@AfterEach`）进行。
- 测试实例生命周期 — `PER_METHOD`，默认行为：



<https://www.arhohuttunen.com/junit-5-test-lifecycle/>

- 为了使每个测试方法在隔离的实例中执行并避免可变实例状态带来的意外副作用，JUnit 在执行每个测试方法之前都会创建该测试类的一个新实例（默认的 `PER_METHOD` 生命周期）。
- 流程：先执行类级别的 `@BeforeAll`（如有），然后为每个测试方法构造新的测试类实例，执行 `@BeforeEach` → 测试方法 → `@AfterEach`，所有测试完成后执行 `@AfterAll`。
- 测试实例生命周期 — `PER_CLASS`：



- 如果希望在同一测试实例上执行所有测试方法，可以在测试类上使用 `@TestInstance(Lifecycle.PER_CLASS)`。
 - 在 `PER_CLASS` 模式下：每个测试类只创建一个测试实例（而不是为每个方法创建新实例）。
 - 若测试方法依赖于实例变量（状态），需要在 `@BeforeEach` 或 `@AfterEach` 中重置这些状态以避免互相影响。
 - 在这种模式下，`@BeforeAll` 和 `@AfterAll` 方法可以为非 `static` 方法（即不强制要求 `static`）。
- 断言 Assertions：
- Assertions 是一组工具方法，用于在测试中断言各种条件是否成立。

```

@Test
void standardAssertions(){
    assertEquals(2, Calculator.add(1,1));

    assertEquals(4, Calculator.multiply(2,2),
        "Optional failure messages");

    assertTrue(Calculator.add(1,1) == 2);

    assertEquals(new int[]{1,2}, new int[]{1,2,3});

    assertNull(null);
}
  
```

- 注意：如果某个断言失败，当前测试方法会立即停止并报失败，后续的断言不会被执行（默认行为）。
- 批量断言 `assertAll`

```
Address address = unitUnderTest.methodUnderTest();
assertAll("Should return address of Oracle's headquarter",
    () -> assertEquals("Redwood Shores", address.getCity()),
    () -> assertEquals("Oracle Parkway", address.getStreet()),
    () -> assertEquals("500", address.getNumber())
);
```

```
org.opentest4j.MultipleFailuresError:
Should return address of Oracle's headquarter (3 failures)
expected: <Redwood Shores> but was: <Walldorf>
expected: <Oracle Parkway> but was: <Dietmar-Hopp-Allee>
expected: <500> but was: <16>
```

- `assertAll` 用于同时执行多条断言并收集所有失败：即使某些断言失败，`assertAll` 也会继续执行其它断言。
- 当有多个断言失败，所有失败会被聚合并以 `MultipleFailuresError` 的形式报告，方便一次性查看所有不通过项。
- 适用场景：需要在一次检查中验证对象的多个属性，并希望看到所有错误而不是第一个失败就中止。
- 前提假设 Assumptions:
 - Assumptions 是一组用于基于条件决定是否执行测试的工具方法。
 - 与断言不同：断言失败算作测试失败；而假设失败会导致测试被“中止/跳过” (aborted)，不会记为失败。
 - 典型用途：当在当前运行环境下继续执行测试没有意义时使用（例如测试依赖某个外部资源或特定运行环境不存在时），可以用假设跳过该测试。

```
private final Calculator calculator = new Calculator();

@Test
void testOnlyOnCiServer() {
    assumeTrue("CI".equals(System.getenv("ENV")));
    // remainder of test
}

@Test
void testOnlyOnDeveloperWorkstation() {
    assumeTrue("DEV".equals(System.getenv("ENV")),
        () -> "Aborting test: not on developer workstation");
    // remainder of test
}

@Test
void testInAllEnvironments() {
    assumingThat("CI".equals(System.getenv("ENV")),
        () -> {
            // perform these assertions only on the CI server
            assertEquals(2, calculator.divide(4, 2));
        });

    // perform these assertions in all environments
    assertEquals(42, calculator.multiply(6, 7));
}
```

- Conditional Test Execution

- Entire test classes or individual test methods may be disabled via the `@Disabled` annotation

```
@Target({ElementType.TYPE, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)
@Documented
@API(
    status = Status.STABLE,
    since = "5.0"
)
public @interface Disabled {
    String value() default "";
}
```

- Developers could either enable or disable a test based on certain conditions programmatically

```
@Test
@EnabledOnOs({ LINUX, MAC })
void onLinuxOrMac() {
    // ...
}

@Test
@DisabledOnOs(WINDOWS)
void notOnWindows() {
    // ...
}
```

```
@Test
@EnabledForJreRange(min = JAVA_9, max = JAVA_11)
void fromJava9to11() {
    // ...
}
```

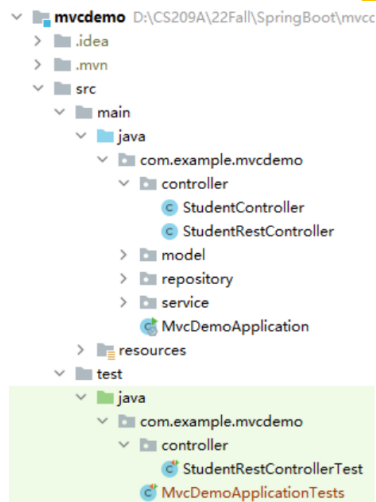
```
@Test
@DisabledForJreRange(min = JAVA_9)
void notFromJava9toCurrentJavaFeatureNumber() {
    // ...
}
```

- 可以通过 `@Disabled` 注解禁用整个测试类或单个测试方法。
- 开发者也可以根据条件以编程方式启用或禁用测试，例如：
 - 基于操作系统启用/禁用（`@EnabledOnOs` / `@DisabledOnOs`）
 - 基于 JRE/Java 版本范围启用/禁用（`@EnabledForJreRange` / `@DisabledForJreRange`）
- 注解既可用于类级别也可用于方法级别，方便根据运行环境控制测试是否执行。
- Parameterized Test

```
@ParameterizedTest
@ValueSource(strings = { "racecar", "radar", "able was I ere I saw elba" })
void palindromes(String candidate) {
    assertTrue(StringUtils.isPalindrome(candidate));
}
```

```
palindromes(String) ✓
└─ [1] candidate=racecar ✓
└─ [2] candidate=radar ✓
└─ [3] candidate=able was I ere I saw elba ✓
```

- 参数化测试允许使用不同参数多次运行同一个测试方法。
- 使用 `@ParameterizedTest` 注解标记参数化测试方法。
- 必须声明至少一个参数源（如 `@ValueSource`、`@CsvSource`、`@MethodSource` 等）为每次调用提供参数，并在测试方法中接收这些参数。
- 适合对相同逻辑使用多组输入进行验证的场景。
- JUnit 测试
- Spring Boot 测试
 - Test 结构



- 健全性检查 (Sanity Check)：对系统或应用的基本功能做简单验证，确保核心功能可用。

```
@SpringBootTest
class MvcDemoApplicationTests {

    @Test
    void contextLoads() {
    }

}
```

- @SpringBootTest 注解会让 Spring Boot 查找主配置类（带 @SpringBootApplication 的类）并使用它启动 Spring 应用上下文用于测试。
- 常见示例：编写一个 contextLoads() 测试方法，确认应用上下文能够成功启动（作为最基本的可用性检查）。
- 在 @SpringBootTest 的测试类中注入控制器（@Autowired StudentController controller），并在 contextLoads() 中断言 controller 非空（如 assertThat(controller).isNotNull()），以确认应用上下文确实创建了该控制器。

```
@SpringBootTest
class MvcDemoApplicationTests {

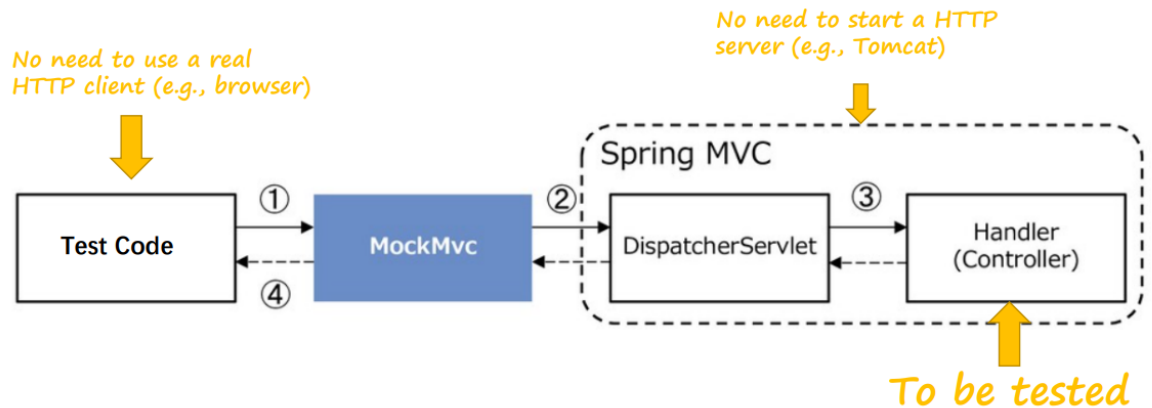
    1 usage
    @Autowired
    private StudentController controller;

    @Test
    void contextLoads() {
        assertThat(controller).isNotNull();
    }

}
```

- 对控制器进行单元测试
 - 目标：对控制器（Web 层）做单元测试，检查：
 - URL 映射是否正确；
 - 能否正确处理传入的 HTTP 请求；
 - 是否正确返回 HTTP 响应。
 - 使用传统方式测试会需要启动一个真实的 Web 服务器、开放端口并用浏览器或 HTTP 客户端访问，这通常：

- 慢且开销大；
- 受网络环境影响。
- Spring MockMvc:



<https://speakerdeck.com/rshindo/spring-fest-2020?slide=34>

- MockMvc 允许在不启动真实 HTTP 服务器和不使用真实浏览器的情况下测试 Spring MVC 控制器。
- 流程概述：
 - 测试代码设置 HTTP 请求的 URL 与参数；
 - MockMvc 模拟 (mock) 一个 HTTP 请求并将其发送到 DispatcherServlet，无需浏览器或启动服务器；
 - DispatcherServlet 调用相应的控制器/处理器；
 - 测试代码使用 MockMvc 断言响应结果。
- 优点：快速、可复现、无需网络环境和服务器开销，适合对控制器行为做精确验证。
- 测试用例示例：

```

@SpringBootTest
@AutoConfigureMockMvc
class StudentRestControllerTest {
    // usages
    @Autowired
    private MockMvc mvc;

    // yidao
    @Test
    void getOneStudent() throws Exception {
        mvc.perform(get( urlTemplate: "/api/students/getOne/{id}", ...UriVars: 1))
            .andExpect(status().isOk())
            .andExpect(jsonPath( expression: "$.name").value( expectedValue: "Mary"))
            .andDo(print());
    }
}
  
```

- @AutoConfigureMockMvc: Spring Boot 的注解，在测试类中自动配置并注入 MockMvc 实例，便于编写 MockMvc 测试。
 - 示例测试要点：
 - 在测试类上使用 @SpringBootTest 与 @AutoConfigureMockMvc；

- 注入 private MockMvc mvc;
- 编写测试方法，使用 mvc.perform(get("/api/students/getOne/{id}", 1)) 发起模拟请求；
- 使用 andExpect(status().isOk())、andExpect(jsonPath("\$.name").value("Mary")) 等断言验证响应状态与 JSON 字段值；
- 结论：结合 @AutoConfigureMockMvc 可方便地编写针对控制器的单元/集成测试，既快速又能验证 HTTP 层的行为。
- 使用 MockMvc.perform() 构造并发起模拟的 HTTP 请求。
 - 使用 MockMvc.andExpect() 断言响应满足预期，例如：
 - 响应状态 (status)
 - 响应内容类型 (content type)
 - 响应体内容 (response body, 例如 JSON 字段值)

```
@Test
void getStudentsByEmail() throws Exception {
    mvc.perform(get( urlTemplate: "/api/students/?email={email}", ...uriVars: "jack")).
        andExpect(status().isOk()).
        andExpect(jsonPath( expression: "$.*", hasSize(1))). // ensure 1 student matches
        andExpect(jsonPath( expression: "$.[0].name").value( expectedValue: "Jack")). // verify
        andDo(print());
}
```

- JsonPath 用于查询和操作 JSON 数据。
- 提供简单直观的语法以访问 JSON 文档中的值（例如 .name、[0].id 等）。
- 在单元测试中可用于断言和验证 JSON 响应内容（结合 MockMvc 的 jsonPath 断言使用）。
- POST 请求与 ObjectMapper

```
@Test
void addOneStudent() throws Exception {
    Student stu = new Student( name: "Jack", email: "jack@mail.com");
    ObjectMapper mapper = new ObjectMapper();

    // convert user to JSON
    String json = mapper.writeValueAsString(stu);

    // send POST request to create user
    mvc.perform(post( urlTemplate: "/api/students/save")
        .contentType(MediaType.APPLICATION_JSON)
        .content(json))
        .andExpect(status().isOk());

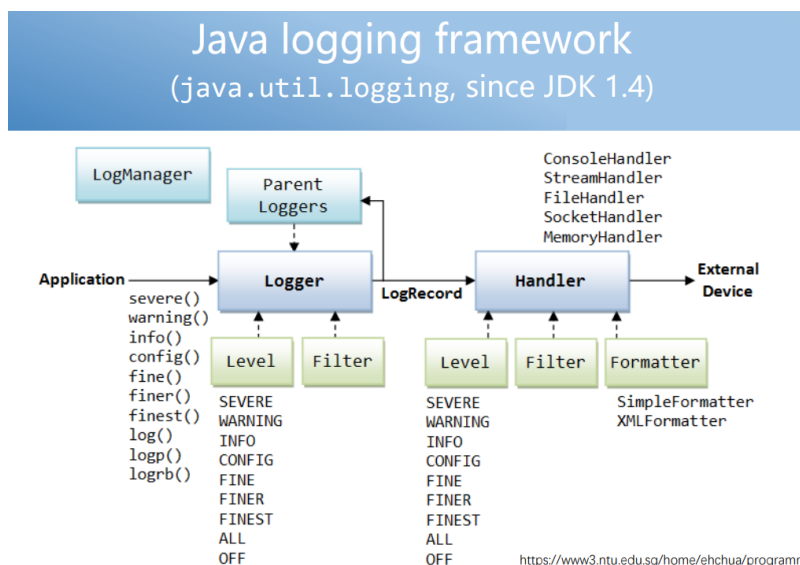
    // verify that user was indeed created
    mvc.perform(get( urlTemplate: "/api/students/getOne/{id}", ...uriVars: 4))
        .andExpect(status().isOk())
        .andExpect(jsonPath( expression: "$.name", is(stu.getName())))
        .andExpect(jsonPath( expression: "$.email", is(stu.getEmail())));
}
```

- ObjectMapper 支持对象与 JSON 字符串之间的相互转换（序列化/反序列化）。
- 可用于测试需要以 JSON 字符串作为请求体的 POST 接口：
 - 将 Java 对象转换为 JSON 字符串（`mapper.writeValueAsString(obj)`），
 - 使用 `mvc.perform(post(...).contentType(APPLICATION_JSON).content(json))` 发送请求，
 - 使用 `andExpect(status().isOk())` 等断言检查响应；
 - 之后可再次通过 GET 验证资源已被创建（并用 `jsonPath` 断言字段值）。

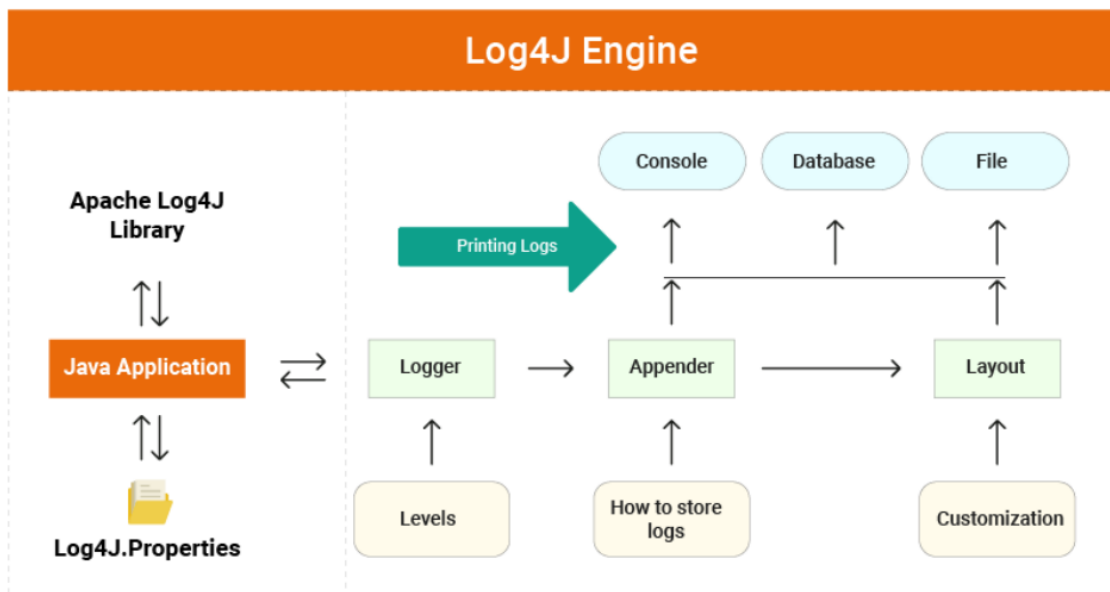
• 日志 Logging

- 日志（Logging）指在程序运行过程中记录信息、事件或错误的过程。通过记录日志可以帮助排错、审计程序行为、监控运行状态以及分析问题原因。
- 日志需求
 - 日志可以有不同的格式：纯文本、XML、HTML 等。
 - 日志可以被不同方式处理：在控制台显示、保存到文件、写入数据库等。
 - 日志应支持不同级别与过滤规则，便于只关注关心的消息。
 - 应用可以针对不同的类或包采用不同的日志机制或配置（灵活性与定向记录）。

• Java 的日志



- Log4J 引擎与 logger 的角色



- Log4J 的基本组成：Logger → Appender → Layout。
 - Logger：负责捕获并处理日志消息（按名称管理）。
 - Appender：负责将日志输出到目标（控制台、文件、数据库等）。
 - 常见用途：在控制台查看、写入日志文件、发送到远程日志服务器等。
 - Layout：负责日志消息的格式化（如何呈现）。
 - 常见格式包括简单文本、HTML、XML 等，便于按需阅读或进一步解析。
- 还有层级（Levels）与如何存储日志（配置文件如 log4j.properties）等可自定义项。
- log4j.properties 配置文件：
 - log4j.properties 是 Log4j 的配置文件，以键值对的形式存储配置项。
 - 该文件包含 Log4j 在运行时所需的全部配置，例如：要使用的 appender（输出器）信息、日志级别配置、以及文件类 appender 的输出文件名等。
 - 通过修改该文件可以动态调整日志行为而无需改动程序代码。

- SLF4J（Simple Logging Facade for Java）是一个轻量的日志门面/抽象层。
 - 应用程序只依赖 SLF4J 接口，而在部署时可以选择绑定到具体的日志实现（如 java.util.logging、Log4j、Logback 等）。
 - 这样能使代码与具体日志框架解耦，便于在不同环境或不同需求下切换实现而无需修改业务代码。

• Spring Boot 的日志

- 在 Spring Boot 中，默认使用 Logback 将 DEBUG 级别的日志输出到控制台。
 - 大多数 starter（例如 spring-boot-starter-web）会依赖 spring-boot-starter-logging，该 starter 会为我们引入 logback（以及 slf4j-api）等相关依赖。
 - 因此在默认配置下，Spring Boot 应用启动后即可看到由 Logback 输出的日志；如果需要，也可以用配置或替换依赖来改用其它日志实现或调整级别与格式。
- 默认情况下，当没有找到任何默认配置文件时，Logback 会向根 Logger 添加一个

ConsoleAppender, 从而把所有日志消息输出到控制台。

- 默认情况下，输出采用 `PatternLayoutEncoder` 格式化。
- `Logger` 的默认日志级别被预设为 `INFO`，这意味着 `TRACE` 和 `DEBUG` 级别的消息默认不可见。
-

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication

public class SpringLoggingDemoApplication {
    private static final Logger LOGGER= LoggerFactory.getLogger(SpringLoggingDemoApplication.class);

    public static void main(String[] args) {
        SpringApplication.run(SpringLoggingDemoApplication.class, args);

        LOGGER.info("Customized INFO log {}", 1);
        LOGGER.debug("Customized DEBUG log {}", 2);
        LOGGER.error("Customized ERROR log {}", 3);
        LOGGER.trace("Customized TRACE log {}", 4);
    }
}
```

Use Logging in Spring Boot

```

W _ _ _ | _ | | | | | | | | | ) ) ) )
_ | _ _ | _ | _ | | | | | | | | / / / /
=====|_|=====|_|_/./././
:: Spring Boot ::                (v3.2.0)

2023-12-12T20:30:59.865+08:00 INFO 24732 --- [           main] c.e.s.SpringLoggingDemoApplication : Starting SpringLogging
2023-12-12T20:30:59.867+08:00 INFO 24732 --- [           main] c.e.s.SpringLoggingDemoApplication : No active profile set,
2023-12-12T20:31:00.679+08:00 INFO 24732 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with
2023-12-12T20:31:00.688+08:00 INFO 24732 --- [           main] o.apache.catalina.core.StandardService : Starting service [Tomc
2023-12-12T20:31:00.688+08:00 INFO 24732 --- [           main] o.apache.catalina.core.StandardEngine : Starting Servlet engine
2023-12-12T20:31:00.757+08:00 INFO 24732 --- [           main] o.a.c.c.C.[Tomcat].[localhost].[/]   : Initializing Spring em
2023-12-12T20:31:00.757+08:00 INFO 24732 --- [           main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationCont
2023-12-12T20:31:01.040+08:00 INFO 24732 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port
2023-12-12T20:31:01.046+08:00 INFO 24732 --- [           main] c.e.s.SpringLoggingDemoApplication : Started SpringLoggin
2023-12-12T20:31:01.048+08:00 INFO 24732 --- [           main] c.e.s.SpringLoggingDemoApplication : Customized INFO log 1
2023-12-12T20:31:01.049+08:00 ERROR 24732 --- [           main] c.e.s.SpringLoggingDemoApplication : Customized ERROR log 3

```

- 展示在代码中如何使用 SLF4J（日志门面）记录日志的示例：
 - 声明 Logger：

```
private static final Logger LOGGER = LoggerFactory.getLogger(YourClass.class);
```
 - 使用不同级别记录：`LOGGER.info(...)`, `LOGGER.debug(...)`, `LOGGER.error(...)`, `LOGGER.trace(...)`。
 - 常见用法是使用带占位符的参数化消息，例如 `LOGGER.info("Customized INFO log {}", 1);`